Validación y Verificación de Software

Clase 4:
Data-Driven Tests

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro Introduction to Software Testing (2nd Edition) y por Manuel Núñez

Testeando isLeapYear...

Testeando isLeapYear...

```
public static boolean isLeapYear(int a) {
  boolean b = false;
  if (((a%4==0) && (a%100!= 0)) || a%400==0)
      b = true;
  return b;
}
```

Testeando isLeapYear...

```java
@Test
 void isLeapYear_ShouldReturnTrueForLeapYear_1() {
     assertTrue(SimpleRoutines.isLeapYear(2000));
 }
 @Test
 void isLeapYear_ShouldReturnTrueForLeapYear_2() {
     assertTrue(SimpleRoutines.isLeapYear(1980));
 }

 @Test
 void isLeapYear_ShouldReturnTrueForLeapYear_3() {
     assertTrue(SimpleRoutines.isLeapYear(1992));
 }
```

Testeando isLeapYear...

```java
@Test
void isLeapYear_ShouldReturnFalseForNonLeapYear_1() {
    assertFalse(SimpleRoutines.isLeapYear(2001));
}
@Test
void isLeapYear_ShouldReturnFalseForNonLeapYear_2() {
    assertFalse(SimpleRoutines.isLeapYear(2023));
}

@Test
void isLeapYear_ShouldReturnFalseForNonLeapYear_3() {
    assertFalse(SimpleRoutines.isLeapYear(1994));
}
```

```java
@Test
 void isLeapYear_ShouldReturn
     assertTrue(SimpleRoutine
 }
 @Test
 void isLeapYear_ShouldReturn
     assertTrue(SimpleRoutine
 }

 @Test
 void isLeapYear_ShouldReturnTrueForLeapYear_3() {
     assertTrue(SimpleRoutines.isLeapYear(1992));
 }
```

Testeando isLeapYear...

```java
@Test
void isLeapYear_ShouldReturnFalseForNonLeapYear_1() {
    assertFalse(SimpleRoutines.isLeapYear(2001));
}
@Test
void isLeapYear_ShouldReturnFalseForNonLeapYear_2() {
    assertFalse(SimpleRoutines.isLeapYear(2023));
}

@Test
void isLeapYear_ShouldReturnFalseForNonLeapYear_3() {
    assertFalse(SimpleRoutines.isLeapYear(1994));
}
```

```java
@Test
 void isLeapYear_ShouldReturn
     assertTrue(SimpleRoutine
 }
 @Test
 void isLeapYear_ShouldReturn
     assertTrue(SimpleRoutine
 }

 @Test
 void isLeapYear_ShouldReturnTrueForLeapYear_3() {
     assertTrue(SimpleRoutines.isLeapYear(1992));
 }
```

Estos tests poseen exactamente la misma estructura, pero difieren en los datos que se utilizan para realizar el arrange del test

Data-Driven Tests

- Problema
  - Cómo evitar la repetición de código?
- Ejemplo simple
  - Chequear que un año particular es bisiesto es lo mismo que chequear que cualquier otro año es bisiesto.
  - Realmente quisieramos escribir un único test
- Data-driven
  - Los mismos test se corren sobre cada conjunto de valores definido.

Tests “parametrizados” por los datos que manipulan

Útil en donde los tests poseen exactamente la misma estructura, pero difieren en los datos que se utilizan para realizar el arrange del test.

separamos su estructura de los datos que manipulan.

Test parametrizados

```java
@ParameterizedTest
@ValueSource(ints = {2000, 1980, 1992})
void isLeapYear_ShouldReturnTrueForLeapYear(int year) {
    assertTrue(SimpleRoutines.isLeapYear(year));
}


@ParameterizedTest
@ValueSource(ints = {2001, 2023, 1994})
void isLeapYear_ShouldReturnFalseForNonLeapYear(int year) {
    assertFalse(SimpleRoutines.isLeapYear(year));
}
```

Test parametrizados

test parametrizados

```
@ParameterizedTest
@ValueSource(ints = {2000, 1980, 1992})
void isLeapYear_ShouldReturnTrueForLeapYear(int year) {
    assertTrue(SimpleRoutines.isLeapYear(year));
}


@ParameterizedTest
@ValueSource(ints = {2001, 2023, 1994})
void isLeapYear_ShouldReturnFalseForNonLeapYear(int year) {
    assertFalse(SimpleRoutines.isLeapYear(year));
}
```

Test parametrizados

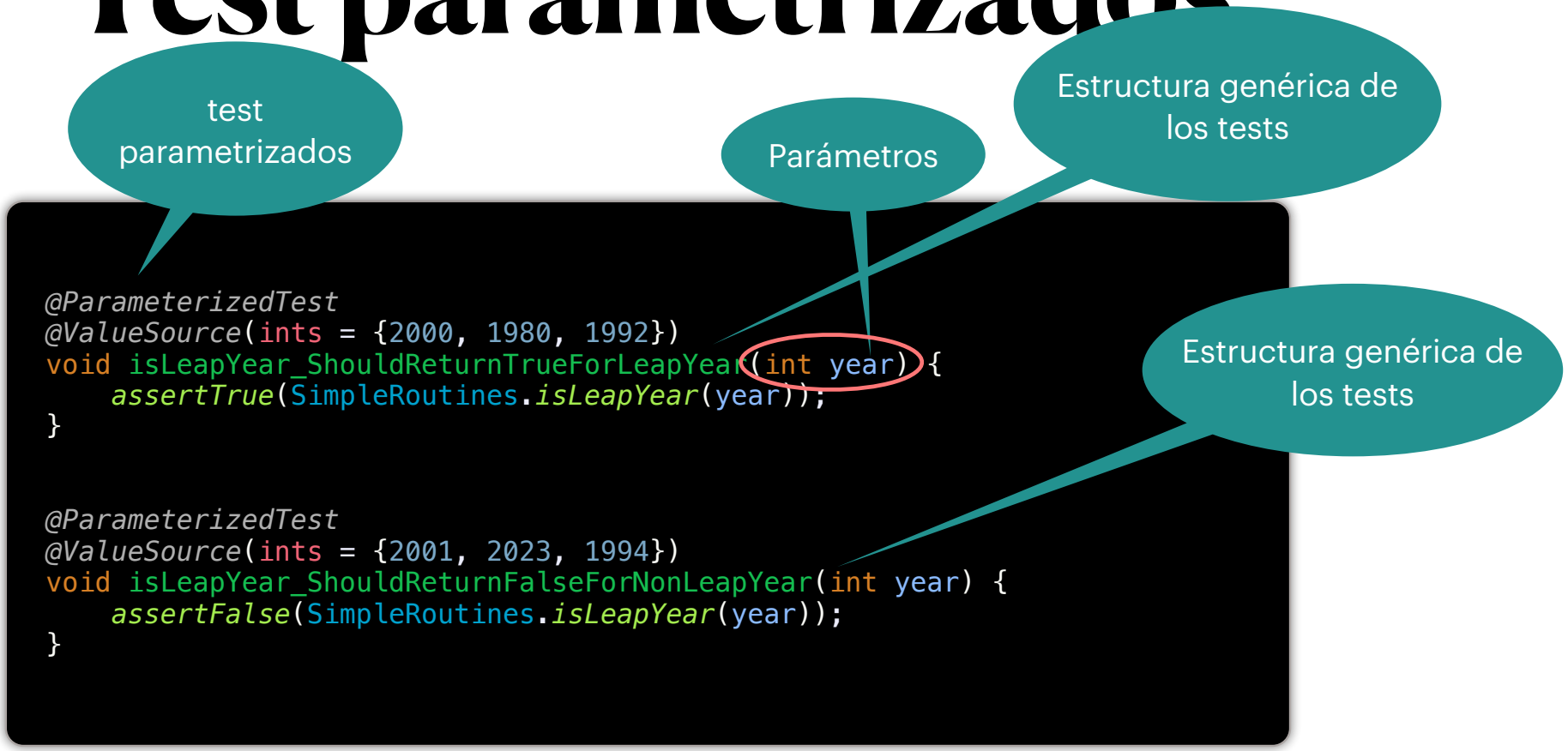
test parametrizados

Parámetros

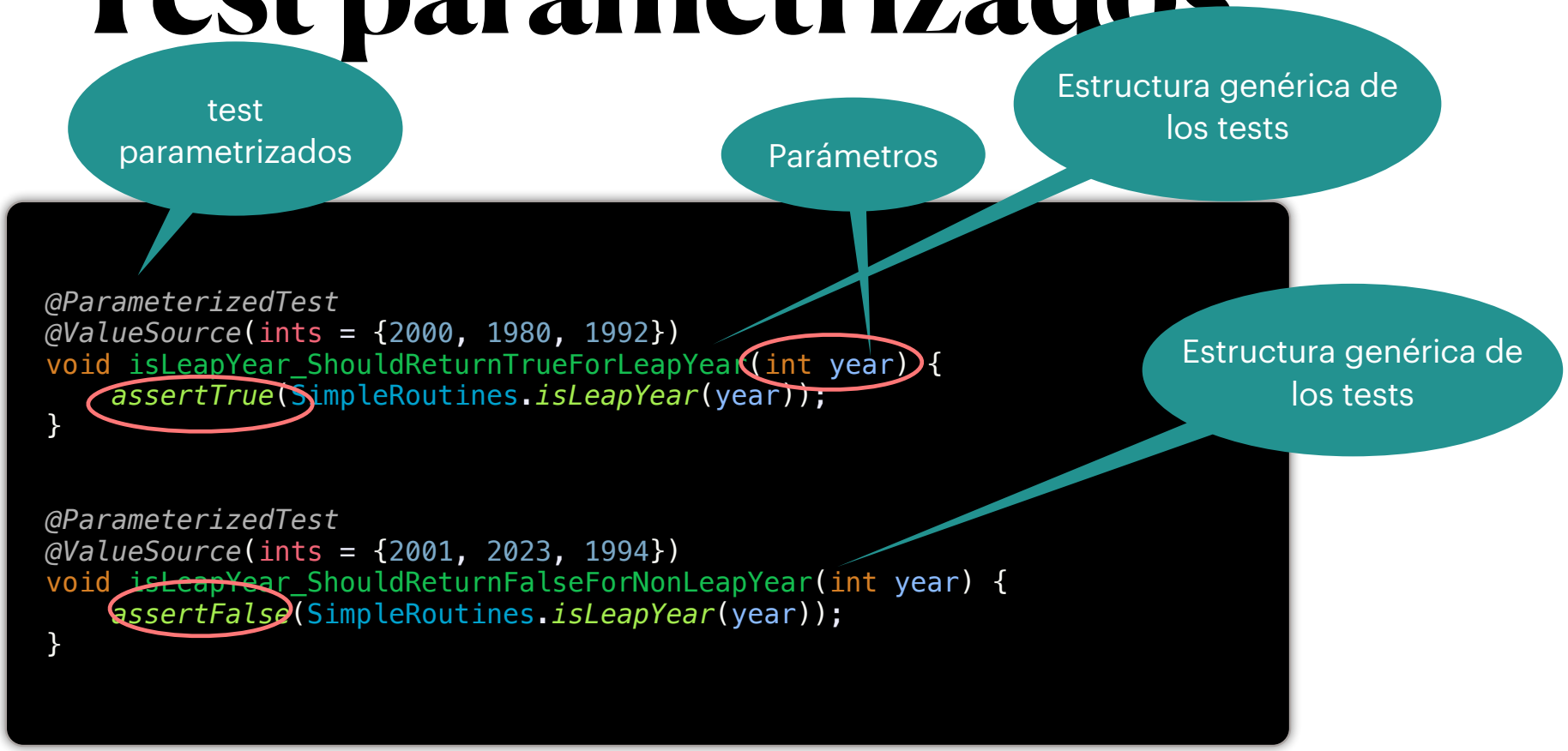
```
@ParameterizedTest
@ValueSource(ints = {2000, 1980, 1992})
void isLeapYear_ShouldReturnTrueForLeapYear(int year) {
    assertTrue(SimpleRoutines.isLeapYear(year));
}


@ParameterizedTest
@ValueSource(ints = {2001, 2023, 1994})
void isLeapYear_ShouldReturnFalseForNonLeapYear(int year) {
    assertFalse(SimpleRoutines.isLeapYear(year));
}
```

Test parametrizados

Test parametrizados



Test parametrizados



Test parametrizados

Test parametrizados

limitaciones de @ValueSource:

Tipos:

- ★ *tipos primitivos* (byte, char, double, float, int, long, or short)
- ★ java.lang.String

Además, podemos pasar un solo argumento.

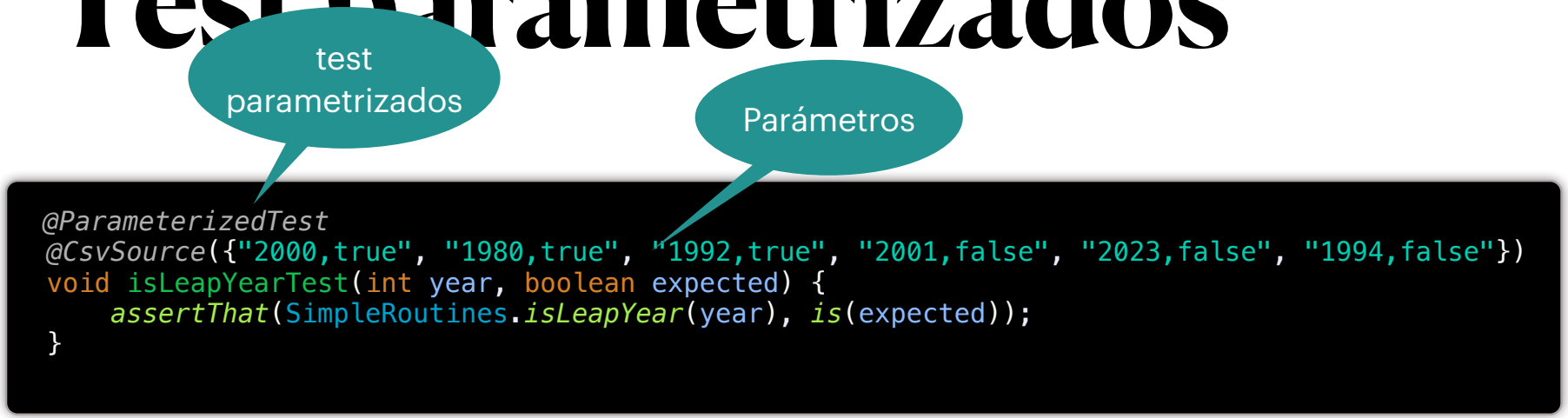
Test parametrizados

```java
@ParameterizedTest
@CsvSource({"2000,true", "1980,true", "1992,true", "2001,false", "2023,false", "1994,false"})
void isLeapYearTest(int year, boolean expected) {
    assertThat(SimpleRoutines.isLeapYear(year), is(expected));
}
```

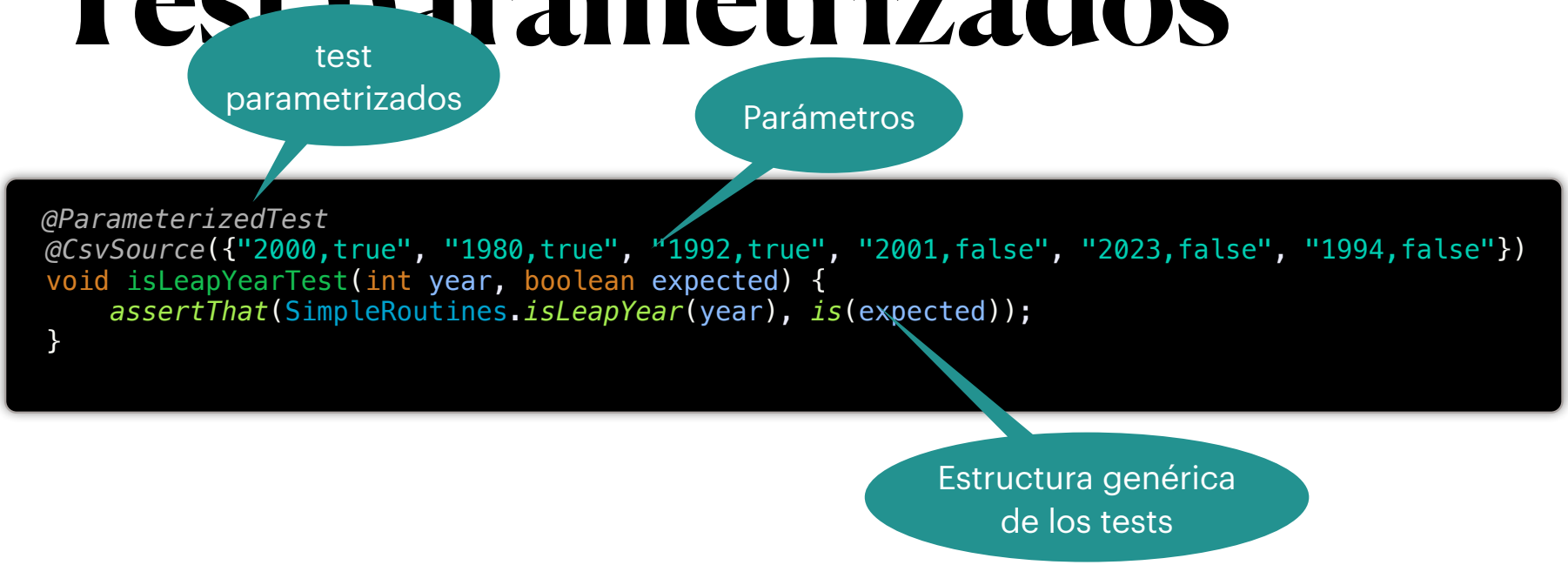
Test parametrizados


```
@ParameterizedTest
@CsvSource({"2000,true", "1980,true", "1992,true", "2001,false", "2023,false", "1994,false"})
void isLeapYearTest(int year, boolean expected) {
    assertThat(SimpleRoutines.isLeapYear(year), is(expected));
}
```

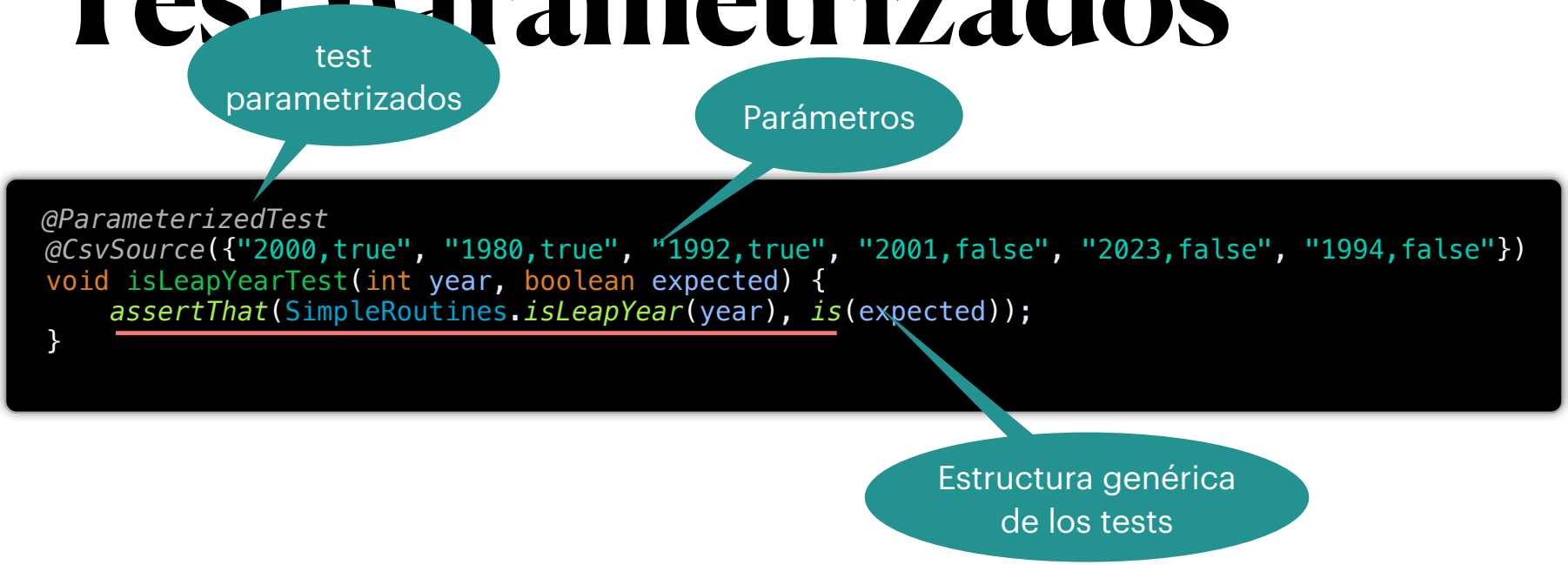
Test parametrizados



Test parametrizados



Test parametrizados



Test parametrizados

test parametrizados

Parámetros

```java
@ParameterizedTest
@CsvSource({"2000,true", "1980,true", "1992,true", "2001,false", "2023,false", "1994,false"})
void isLeapYearTest(int year, boolean expected) {
    assertThat(SimpleRoutines.isLeapYear(year), is(expected));
}
```

Estructura genérica de los tests

```java
@ParameterizedTest
@CsvFileSource(resources = "leapYear.csv", numLinesToSkip = 1)
void isLeapYearTest_2(int year, boolean expected) {
    assertThat(SimpleRoutines.isLeapYear(year), is(expected));
}
```

Test parametrizados

test parametrizados

Parámetros

```
@ParameterizedTest
@CsvSource({"2000,true", "1980,true", "1992,true", "2001,false", "2023,false", "1994,false"})
void isLeapYearTest(int year, boolean expected) {
    assertThat(SimpleRoutines.isLeapYear(year), is(expected));
}
```

Estructura genérica de los tests

```
@ParameterizedTest
@CsvFileSource(resources = "leapYear.csv", numLinesToSkip = 1)
void isLeapYearTest_2(int year, boolean expected) {
    assertThat(SimpleRoutines.isLeapYear(year), is(expected));
}
```

Year	Expected
2000	true
1980	true
1992	true
2001	false
2023	false
1994	false

Test parametrizados: un ejemplo

```java
@ParameterizedTest
@CsvSource({"test,e,1", "valeria,a,2", "otorrinolaringologo,o,6"})
void ocurrencesCountTest(String someString, char someChar, int expected) {
        int result = Sorting.ocurrencesCount(someString, someChar);
        assertThat(result, equalTo(expected));
}
```

Test parametrizados: un ejemplo

```
@ParameterizedTest
@CsvSource({"test,e,1", "valeria,a,2", "otorrinolaringologo,o,6"})
void ocurrencesCountTest(String someString, char someChar, int expected) {
        int result = Sorting.ocurrencesCount(someString, someChar);
        assertThat(result, equalTo(expected));
}
```

```
@ParameterizedTest
@CsvFileSource(resources = "occurrenceCount.csv", numLinesToSkip = 1)
  void ocurrencesCountTest(String someString, char someChar, int expected) {
        int result = Sorting.ocurrencesCount(someString, someChar);
        assertThat(result, equalTo(expected));
}
```

Test parametrizados: un ejemplo

```java
@ParameterizedTest
@CsvSource({"test,e,1", "valeria,a,2", "otorrinolaringologo,o,6"})
void ocurrencesCountTest(String someString, char someChar, int expected) {
        int result = Sorting.ocurrencesCount(someString, someChar);
        assertThat(result, equalTo(expected));
}
```

```java
@ParameterizedTest
@CsvFileSource(resources = "occurrenceCount.csv", numLinesToSkip = 1)
  void ocurrencesCountTest(String someString, char someChar, int expected) {
        int result = Sorting.ocurrencesCount(someString, someChar);
        assertThat(result, equalTo(expected));
}
```

Word	Letter	Expected
test	e	1
vale	n	0
...	....	...
...	...	...

Conversión de tipos

Tipo	Ejemplo
Boolean/boolean	“true” => true
Character/char	“o” => o
Integer/int	“15” => 15
Float/float	“1.0” => 1.0f
Double/double	“1.0” => 1.0d
....	...

Conversión de tipos

Pero JUnit no sabe convertir todos los tipos desde Strings. Por ejemplo, cuando los tests toman arreglos

Conversión de tipos

Pero JUnit no sabe convertir todos los tipos desde Strings. Por ejemplo, cuando los tests toman arreglos

```
@Test
public void testBubleSort00() {
   int [] array = {7,6,5};
   int [] sortedarray = {5,6,7};
   Sorting.bubbleSort(array);
   assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
}


@Test
public void testBubleSort01() {
   int [] array = {7,6,5,4};
   int [] sortedarray = {4,5,6,7};
   Sorting.bubbleSort(array);
   assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
}

@Test
public void testBubleSort02() {
   int [] array = {0,7,6,5,4};
   int [] sortedarray = {0,4,5,6,7};
   Sorting.bubbleSort(array);
   assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
}
```

Test parametrizados

```java
public class BubbleSortParamTest {

  private static Stream<Arguments> arraysProvider() {
      return Stream.of(
        Arguments.of(new int [] { 7,8,9 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,8,7 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 7,9,8 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,7,9,8 }, new int [] { 7,8,9,9 }) ,
        Arguments.of(new int [] { 1 }, new int [] { 1 }),
        Arguments.of(new int [] {  }, new int [] {  }) ,
        Arguments.of(new int [] { -9,-8,-7 }, new int [] { -9,-8,-7 })
       );
  }

  @ParameterizedTest(name = "{index}: sorting( {0} ) = {1}")
  @MethodSource("arraysProvider")
  public void testBubleSort(int [] array, int [] sortedarray) {
    Sorting.bubbleSort(array);
    assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
  }


}
```

Test parametrizados

```java
public class BubbleSortParamTest {

  private static Stream<Arguments> arraysProvider() {
      return Stream.of(
        Arguments.of(new int [] { 7,8,9 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,8,7 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 7,9,8 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,7,9,8 }, new int [] { 7,8,9,9 }) ,
        Arguments.of(new int [] { 1 }, new int [] { 1 }),
        Arguments.of(new int [] {  }, new int [] {  }) ,
        Arguments.of(new int [] { -9,-8,-7 }, new int [] { -9,-8,-7 })
       );
  }

  @ParameterizedTest(name = "{index}: sorting( {0} ) = {1}")
  @MethodSource("arraysProvider")
  public void testBubleSort(int [] array, int [] sortedarray) {
    Sorting.bubbleSort(array);
    assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
  }


}
```

Test parametrizados

```java
public class BubbleSortParamTest {

  private static Stream<Arguments> arraysProvider(){
      return Stream.of(
        Arguments.of(new int [] { 7,8,9 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,8,7 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 7,9,8 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,7,9,8 }, new int [] { 7,8,9,9 }) ,
        Arguments.of(new int [] { 1 }, new int [] { 1 }),
        Arguments.of(new int [] {  }, new int [] {  }) ,
        Arguments.of(new int [] { -9,-8,-7 }, new int [] { -9,-8,-7 })
       );
  }

  @ParameterizedTest(name = "{index}: sorting( {0} ) = {1}")
  @MethodSource("arraysProvider")
  public void testBubleSort(int [] array, int [] sortedarray) {
    Sorting.bubbleSort(array);
    assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
  }


}
```

Test parametrizados

```java
public class BubbleSortParamTest {

  private static Stream<Arguments> arraysProvider(){
      return Stream.of(
        Arguments.of(new int [] { 7,8,9 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,8,7 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 7,9,8 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,7,9,8 }, new int [] { 7,8,9,9 }) ,
        Arguments.of(new int [] { 1 }, new int [] { 1 }),
        Arguments.of(new int [] {  }, new int [] {  }) ,
        Arguments.of(new int [] { -9,-8,-7 }, new int [] { -9,-8,-7 })
       );
  }

  @ParameterizedTest(name = "{index}: sorting( {0} ) = {1}")
  @MethodSource("arraysProvider")
  public void testBubleSort(int [] array, int [] sortedarray) {
    Sorting.bubbleSort(array);
    assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
  }


}
```

Estructura genérica de los tests

Test parametrizados

```java
public class BubbleSortParamTest {

  private static Stream<Arguments> arraysProvider(){
      return Stream.of(
        Arguments.of(new int [] { 7,8,9 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,8,7 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 7,9,8 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,7,9,8 }, new int [] { 7,8,9,9 }) ,
        Arguments.of(new int [] { 1 }, new int [] { 1 }),
        Arguments.of(new int [] {  }, new int [] {  }) ,
        Arguments.of(new int [] { -9,-8,-7 }, new int [] { -9,-8,-7 })
       );
  }

  @ParameterizedTest(name = "{index}: sorting( {0} ) = {1}")
  @MethodSource("arraysProvider")
  public void testBubleSort(int [] array, int [] sortedarray) {
    Sorting.bubbleSort(array);
    assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
  }


}
```

Generador de parámetros

Estructura genérica de los tests

Test parametrizados

```java
public class BubbleSortParamTest {

  private static Stream<Arguments> arraysProvider(){
      return Stream.of(
        Arguments.of(new int [] { 7,8,9 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,8,7 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 7,9,8 }, new int [] { 7,8,9 }),
        Arguments.of(new int [] { 9,7,9,8 }, new int [] { 7,8,9,9 }) ,
        Arguments.of(new int [] { 1 }, new int [] { 1 }),
        Arguments.of(new int [] {  }, new int [] {  }) ,
        Arguments.of(new int [] { -9,-8,-7 }, new int [] { -9,-8,-7 })
       );
  }

  @ParameterizedTest(name = "{index}: sorting( {0} ) = {1}")
  @MethodSource("arraysProvider")
  public void testBubleSort(int [] array, int [] sortedarray) {
    Sorting.bubbleSort(array);
    assertArrayEquals("El arreglo esta desordenado", sortedarray, array);
  }


}
```

Generador de parámetros

Cada tupla de valores es un test individual

Estructura genérica de los tests

Test parametrizados

```
private static Stream<Arguments> findLastArgsProvider() {
    return Stream.of(
        Arguments.of(new int[] {2,3,5}, 2, 0),
        Arguments.of(new int[] {1,3,1}, 1, 2),
        Arguments.of(new int[] {1,2,5}, 3, -1)
    );
  }



  @ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
  @MethodSource("findLastArgsProvider")
  void findLastParamTest(int [] arr, int elem, int expected) {
    int result = SimpleRoutines.findLast(arr, elem);
     assertEquals(expected, result);
  }
```

Test parametrizados

```
private static Stream<Arguments> findLastArgsProvider() {
    return Stream.of(
        Arguments.of(new int[] {2,3,5}, 2, 0),
        Arguments.of(new int[] {1,3,1}, 1, 2),
        Arguments.of(new int[] {1,2,5}, 3, -1)
    );
  }



  @ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
  @MethodSource("findLastArgsProvider")
  void findLastParamTest(int [] arr, int elem, int expected) {
    int result = SimpleRoutines.findLast(arr, elem);
      assertEquals(expected, result);
  }
```

Estructura genérica de los tests

Test parametrizados

```java
private static Stream<Arguments> findLastArgsProvider() {
    return Stream.of(
        Arguments.of(new int[] {2,3,5}, 2, 0),
        Arguments.of(new int[] {1,3,1}, 1, 2),
        Arguments.of(new int[] {1,2,5}, 3, -1)
    );
  }


  @ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
  @MethodSource("findLastArgsProvider")
  void findLastParamTest(int [] arr, int elem, int expected) {
    int result = SimpleRoutines.findLast(arr, elem);
      assertEquals(expected, result);
  }
```

Generador de parámetros

Estructura genérica de los tests

Test parametrizados

```java
private static Stream<Arguments> findLastArgsProvider() {
    return Stream.of(
        Arguments.of(new int[] {2,3,5}, 2, 0),
        Arguments.of(new int[] {1,3,1}, 1, 2),
        Arguments.of(new int[] {1,2,5}, 3, -1)
    );
}

@ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
@MethodSource("findLastArgsProvider")
void findLastParamTest(int [] arr, int elem, int expected) {
    int result = SimpleRoutines.findLast(arr, elem);
     assertEquals(expected, result);
}
```

Generador de parámetros

Cada tupla de valores es un test individual

Estructura genérica de los tests

Test parametrizados

```java
private static Stream<Arguments> findLastArgsProvider() {
    return Stream.of(
        Arguments.of(new int[] {2,3,5}, 2, 0),
        Arguments.of(new int[] {1,3,1}, 1, 2),
        Arguments.of(new int[] {1,2,5}, 3, -1)
    );
}

@ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
@MethodSource("findLastArgsProvider")
void findLastParamTest(int [] arr, int elem, int expected) {
    int result = SimpleRoutines.findLast(arr, elem);
    assertEquals(expected, result);
}
```

Generador de parámetros

Cada tupla de valores es un test individual

Estructura genérica de los tests

Test parametrizados

```
private static Stream<Arguments> findLastArgsProvider() {
    return Stream.of(
        Arguments.of(new int[] {2,3,5}, 2, 0),
        Arguments.of(new int[] {1,3,1}, 1, 2),
        Arguments.of(new int[] {1,2,5}, 3, -1)
    );
  }


  @ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
  @MethodSource("findLastArgsProvider")
  void findLastParamTest(int [] arr, int elem, int expected) {
    int result = SimpleRoutines.findLast(arr, elem);
      assertEquals(expected, result);
  }
```

Generador de parámetros

Cada tupla de valores es un test individual

Valor esperado

Estructura genérica de los tests

Test parametrizados

```
  @ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
  @CsvSource({"'[1,2,3]', 3, 2", "'[1,3,3]', 3, 2", "'[1,2,3]', 0, -1"})
   void findLastParamTest_1(@ConvertWith(ArrayFromString.class) int [] arr, int elem, int expected){
   int result = SimpleRoutines.findLast(arr, elem); assertEquals(expected, result);

}
```

```
import org.junit.jupiter.params.converter.ArgumentConversionException;
import org.junit.jupiter.params.converter.SimpleArgumentConverter;

public class ArrayFromString extends SimpleArgumentConverter{

  @Override
  protected Object convert(Object source, Class<?> targetType) throws ArgumentConversionException {

    String strSource = ((String)source);
    strSource = ((String)source).substring(1, strSource.length()-1);
    String [] strArray = strSource.split(",");
    int [] result = new int [strArray.length];
    for (int i = 0; i < strArray.length; i++)
      result[i] = Integer.parseInt(strArray[i]);
    return result;

  }
```

Test parametrizados

```
  @ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
  @CsvSource({"'[1,2,3]', 3, 2", "'[1,3,3]', 3, 2", "'[1,2,3]', 0, -1"})
   void findLastParamTest_1(@ConvertWith(ArrayFromString.class) int [] arr, int elem, int expected){
   int result = SimpleRoutines.findLast(arr, elem); assertEquals(expected, result);

}
```

```
import org.junit.jupiter.params.converter.ArgumentConversionException;
import org.junit.jupiter.params.converter.SimpleArgumentConverter;

public class ArrayFromString extends SimpleArgumentConverter{

  @Override
  protected Object convert(Object source, Class<?> targetType) throws ArgumentConversionException {

    String strSource = ((String)source);
    strSource = ((String)source).substring(1, strSource.length()-1);
    String [] strArray = strSource.split(",");
    int [] result = new int [strArray.length];
    for (int i = 0; i < strArray.length; i++)
      result[i] = Integer.parseInt(strArray[i]);
    return result;

  }
```

Test parametrizados

```
  @ParameterizedTest(name = "{index}: last {1} in {0} is in position {2}")
  @CsvSource({"'[1,2,3]', 3, 2", "'[1,3,3]', 3, 2", "'[1,2,3]', 0, -1"})
   void findLastParamTest_1(@ConvertWith(ArrayFromString.class) int [] arr, int elem, int expected){
   int result = SimpleRoutines.findLast(arr, elem); assertEquals(expected, result);

}
```

```
import org.junit.jupiter.params.converter.ArgumentConversionException;
import org.junit.jupiter.params.converter.SimpleArgumentConverter;

public class ArrayFromString extends SimpleArgumentConverter{

  @Override
  protected Object convert(Object source, Class<?> targetType) throws ArgumentConversionException {

    String strSource = ((String)source);
    strSource = ((String)source).substring(1, strSource.length()-1);
    String [] strArray = strSource.split(",");
    int [] result = new int [strArray.length];
    for (int i = 0; i < strArray.length; i++)
      result[i] = Integer.parseInt(strArray[i]);
    return result;

  }
```

Lectura Recomendada

Guía de usuario de JUnit 5:

- ★ https://junit.org/junit5/docs/current/user-guide/#writing-tests-parameterized-tests